SERVA

Step by Step Build Guide

Universal Booking & Payments Marketplace

MVP Stack: React Native + Expo, Next.js, Supabase, Stripe

For non-technical founders learning to code

April 2026

PART 1: ENVIRONMENT SETUP (Steps 1-15)

Set up your development environment with all necessary tools, accounts, and project structure. This is the foundation for building Serva.

Step 1: Install Node.js

Node.js is the JavaScript runtime that powers both your backend server and build tools. Download the LTS (Long Term Support) version.

Terminal Command:

```
# Visit https://nodejs.org and download Node.js LTS (v18 or v20) # Verify installation: node --version npm --version
```

You'll know it's working when...

- npm and node commands work from terminal
- Versions are v18+ for node, npm v8+

Common Issues:

Command not found:

- Add Node.js to PATH or restart terminal after installation

Step 2: Install Git

Git is version control software that tracks code changes and enables collaboration. Essential for pushing code to GitHub.

Terminal Command:

```
# macOS with Homebrew: brew install git  # Windows/Linux: download from https://git-scm.com  # Verify: git --version
```

You'll know it's working when...

- git --version returns v2.30+

Common Issues:

Command not found:

- Restart terminal or add Git to PATH

Step 3: Install VS Code or Cursor

A code editor is where you'll write your code. VS Code is free; Cursor is a newer AI-powered alternative ($20/month).

Terminal Command:

```
# VS Code: https://code.visualstudio.com # Cursor: https://cursor.com # Download and install the version for your OS
```

You'll know it's working when...

- Code editor launches and opens successfully
- Extensions can be installed (see next step)

Step 4: Install Recommended VS Code Extensions

Extensions supercharge your editor with autocompletion, linting, and formatting for React, TypeScript, and databases.

You'll know it's working when...

- Install: ESLint, Prettier, Thunder Client (API testing), SQLTools, Expo Tools, React Native Tools

Step 5: Create GitHub Account & SSH Key

GitHub hosts your code repository in the cloud. SSH keys authenticate your local machine to GitHub without passwords.

Terminal Command:

```
# 1. Create account: https://github.com/signup # 2. Generate SSH key: ssh-keygen -t ed25519 -C 'your@email.com' # 3. Press Enter for default location # 4. Copy public key: cat ~/.ssh/id_ed25519.pub # 5. Go to GitHub Settings > SSH Keys > New > paste key
```

You'll know it's working when...

- SSH key is added to GitHub
- You can run: ssh -T git@github.com (no password needed)

Common Issues:

Permission denied (publickey):

- Ensure SSH key is added to GitHub and agent has key loaded

Step 6: Create Supabase Project

Supabase provides PostgreSQL database, authentication, real-time subscriptions, and file storage—all backend-as-a-service.

Terminal Command:

```
# 1. Go to https://supabase.com and sign up # 2. Create new project # 3. Choose region (same as deployment region for speed) # 4. Set strong password, save to password manager # 5. Wait for project initialization (2-3 minutes) # 6. Go to Settings > API Keys > Copy anon key and service_role key
```

You'll know it's working when...

- Supabase project dashboard is accessible
- You have anon and service_role API keys saved

Step 7: Create Stripe Account & Connect

Stripe processes payments. Stripe Connect lets businesses receive payouts directly to their bank accounts.

Terminal Command:

```
# 1. Go to https://dashboard.stripe.com/register # 2. Sign up with email # 3. Create account as platform (select 'I want to accept payments') # 4. Enable Stripe Connect (Settings > Connect > Activate) # 5. Copy publishable key (starts with pk_) and secret key (starts with sk_)
```

You'll know it's working when...

- Stripe dashboard loads
- You have publishable and secret keys

Common Issues:

Stripe Connect not available:

- Ensure you're on a paid Stripe account or contact support

Step 8: Create Vercel Account

Vercel hosts your Next.js web app with automatic deployments from GitHub. Free tier is sufficient for MVP.

Terminal Command:

```
# 1. Go to https://vercel.com/signup # 2. Sign up with GitHub # 3. Grant Vercel access to your GitHub account # 4. You're ready to deploy
```

You'll know it's working when...

- Vercel dashboard is accessible
- GitHub is connected

Step 9: Create Resend Account

Resend sends transactional emails (confirmations, reminders) with a simple API. Free tier allows 100 emails/day.

Terminal Command:

```
# 1. Go to https://resend.com # 2. Sign up with email # 3. Verify email # 4. Create API key (Settings > API Keys) # 5. Copy and save the key
```

You'll know it's working when...

- Resend API key is saved
- You can send test emails

Step 10: Create Twilio Account

Twilio sends SMS notifications and reminders. Free trial gives you $15 credit.

Terminal Command:

```
# 1. Go to https://www.twilio.com/try-twilio # 2. Sign up and verify phone number # 3. Save Account SID and Auth Token (Settings > Account Info) # 4. Get a Twilio phone number (Phone Numbers > Get Started)
```

You'll know it's working when...

- Twilio account is active
- Account SID, Auth Token, and phone number are saved

Step 11: Enroll Apple Developer & Google Play

These are required to test and publish your mobile app on iOS and Android.

Terminal Command:

```
# Apple Developer (iOS): # 1. Go to https://developer.apple.com/enroll # 2. Cost: $99/year # 3. Create Apple ID or use existing  # Google Play (Android): # 1. Go to https://play.google.com/console # 2. Cost: $25 (one-time) # 3. Create Google account or use existing
```

You'll know it's working when...

- Both accounts are enrolled
- You have Apple and Google accounts linked

Step 12: Install Expo CLI & Expo Go

Expo CLI is the command-line tool for React Native development. Expo Go is the companion app to test on your phone.

Terminal Command:

```
# Install Expo CLI: npm install -g expo-cli  # Verify: expo --version  # Download Expo Go app on your iPhone or Android from App Store / Play Store
```

You'll know it's working when...

- expo --version returns v5+
- Expo Go app is installed on your phone

Step 13: Create GitHub Monorepo & Initial Structure

A monorepo keeps your web, mobile, and shared code in one repository for easier management.

Terminal Command:

```
# 1. Create repo on GitHub (no README/gitignore yet) # 2. Clone to your computer: git clone git@github.com:YOUR-USERNAME/serva.git cd serva  # 3. Create folder structure: mkdir -p apps/web apps/mobile packages/shared touch .gitignore README.md
```

You'll know it's working when...

- Folder structure exists locally
- Git clone works without password (SSH is configured)

Step 14: Initialize npm Workspaces

npm workspaces allow multiple npm packages in one repo, sharing dependencies and enabling local imports.

Terminal Command:

```
# Create root package.json: npm init -y  # Edit package.json to add workspaces: # Change name to 'serva-workspace' # Add: "workspaces": ["apps/web", "apps/mobile", "packages/shared"]
```

Code Example:

// package.json

```
{   "name": "serva-workspace",   "version": "1.0.0",   "workspaces": ["apps/web", "apps/mobile", "packages/shared"],   "devDependencies": {     "typescript": "^5.3.0" } }
```

You'll know it's working when...

- package.json has workspaces array
- Root npm install works

Step 15: Push Initial Code to GitHub

Save your initial structure to GitHub so you have version history and can enable CI/CD later.

Terminal Command:

```
# In the serva folder: git add -A git config user.email 'your@email.com' git config user.name 'Your Name' git commit -m 'Initial monorepo structure' git push -u origin main
```

You'll know it's working when...

- Code appears on GitHub repo page
- No merge conflicts

Common Issues:

Push rejected:

- Ensure main branch exists and you have write access

PART 2: DATABASE SETUP (Steps 16-28)

Build the PostgreSQL schema in Supabase. This is the structure that stores all data: users, businesses, bookings, payments, etc.

Step 16: Create Core Tables (Users, Profiles, Businesses)

The foundation tables store user accounts, profile information, and business data.

Terminal Command:

```
-- In Supabase SQL Editor, run: CREATE TABLE public.users (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   email TEXT UNIQUE NOT NULL,   created_at TIMESTAMP DEFAULT NOW() );  CREATE TABLE public.user_profiles (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   user_id UUID REFERENCES users(id) ON DELETE CASCADE,   first_name TEXT,   last_name TEXT,   phone TEXT,   avatar_url TEXT,   created_at TIMESTAMP DEFAULT NOW() );  CREATE TABLE public.businesses (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   owner_id UUID REFERENCES users(id) NOT NULL,   name TEXT NOT NULL,   description TEXT,   address TEXT,   city TEXT,   state TEXT,   zip_code TEXT,   latitude DECIMAL(10, 8),   longitude DECIMAL(11, 8),   phone TEXT,   website TEXT,   logo_url TEXT,   stripe_account_id TEXT,   is_approved BOOLEAN DEFAULT FALSE,   created_at TIMESTAMP DEFAULT NOW() );
```

You'll know it's working when...

- Tables appear in Supabase UI under Tables
- No errors in SQL editor

Step 17: Create Photos & Services Tables

Store photos for businesses and define the services each business offers.

Terminal Command:

```
CREATE TABLE public.business_photos (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   business_id UUID REFERENCES businesses(id) ON DELETE CASCADE,   photo_url TEXT NOT NULL,   display_order INT,   created_at TIMESTAMP DEFAULT NOW() );  CREATE TABLE public.services (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   business_id UUID REFERENCES businesses(id) ON DELETE CASCADE,   name TEXT NOT NULL,   description TEXT,   duration_minutes INT DEFAULT 60,   price DECIMAL(10, 2) NOT NULL,   is_active BOOLEAN DEFAULT TRUE,   created_at TIMESTAMP DEFAULT NOW() );
```

You'll know it's working when...

- Both tables exist
- Services references businesses correctly

Step 18: Create Staff & Availability Tables

Staff are team members who provide services. Availability defines when they work.

Terminal Command:

```
CREATE TABLE public.staff (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   business_id UUID REFERENCES businesses(id) ON DELETE CASCADE,   name TEXT NOT NULL,   email TEXT,   phone TEXT,   is_active BOOLEAN DEFAULT TRUE,   created_at TIMESTAMP DEFAULT NOW() );  CREATE TABLE public.staff_services (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   staff_id UUID REFERENCES staff(id) ON DELETE CASCADE,   service_id UUID REFERENCES services(id) ON DELETE CASCADE,   UNIQUE(staff_id, service_id) );  CREATE TABLE public.availability_rules (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   staff_id UUID REFERENCES staff(id) ON DELETE CASCADE,   day_of_week INT, -- 0-6 (Sunday-Saturday)   start_time TIME,   end_time TIME,   slots_per_hour INT DEFAULT 2,   created_at TIMESTAMP DEFAULT NOW() );
```

You'll know it's working when...

- All three tables created
- Foreign keys are correct

Step 19: Create Bookings & Payment Tables

Bookings are the core transaction: customer reserves a service slot. Payments track payment status.

Terminal Command:

```
CREATE TABLE public.bookings (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   customer_id UUID REFERENCES users(id) NOT NULL,   service_id UUID REFERENCES services(id) NOT NULL,   staff_id UUID REFERENCES staff(id),   business_id UUID REFERENCES businesses(id) NOT NULL,   scheduled_start TIMESTAMP NOT NULL,   duration_minutes INT,   status TEXT DEFAULT 'pending', -- pending, confirmed, completed, cancelled   customer_name TEXT,   customer_email TEXT,   customer_phone TEXT,   total_price DECIMAL(10, 2),   notes TEXT,   created_at TIMESTAMP DEFAULT NOW() );  CREATE TABLE public.booking_status_history (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   booking_id UUID REFERENCES bookings(id) ON DELETE CASCADE,   old_status TEXT,   new_status TEXT,   changed_at TIMESTAMP DEFAULT NOW() );  CREATE TABLE public.payments (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   booking_id UUID REFERENCES bookings(id) NOT NULL,   stripe_payment_intent_id TEXT UNIQUE,   amount DECIMAL(10, 2) NOT NULL,   status TEXT DEFAULT 'pending', -- pending, succeeded, failed   payout_account_id TEXT,   created_at TIMESTAMP DEFAULT NOW() );
```

You'll know it's working when...

- All three tables exist
- Foreign keys reference bookings and users correctly

Step 20: Create Reviews & Notifications Tables

Customers review services. Notifications alert users about bookings, reminders, and updates.

Terminal Command:

```
CREATE TABLE public.reviews (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   booking_id UUID REFERENCES bookings(id) UNIQUE,   business_id UUID REFERENCES businesses(id) NOT NULL,   customer_id UUID REFERENCES users(id) NOT NULL,   rating INT CHECK (rating >= 1 AND rating <= 5),   comment TEXT,   created_at TIMESTAMP DEFAULT NOW() );  CREATE TABLE public.review_responses (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   review_id UUID REFERENCES reviews(id) ON DELETE CASCADE,   business_owner_id UUID REFERENCES users(id),   response_text TEXT,   created_at TIMESTAMP DEFAULT NOW() );  CREATE TABLE public.notifications (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   user_id UUID REFERENCES users(id) ON DELETE CASCADE,   type TEXT, -- booking_confirmed, reminder, review_request   title TEXT,   message TEXT,   related_booking_id UUID REFERENCES bookings(id),   is_read BOOLEAN DEFAULT FALSE,   created_at TIMESTAMP DEFAULT NOW() );
```

You'll know it's working when...

- Review tables created
- Notifications table links to users

Step 21: Create Form & Favorites Tables

Intake forms collect custom information from customers. Favorites let customers save businesses.

Terminal Command:

```
CREATE TABLE public.form_templates (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   business_id UUID REFERENCES businesses(id) ON DELETE CASCADE,   name TEXT NOT NULL,   description TEXT,   is_required BOOLEAN DEFAULT FALSE,   created_at TIMESTAMP DEFAULT NOW() );  CREATE TABLE public.form_fields (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   form_id UUID REFERENCES form_templates(id) ON DELETE CASCADE,   field_name TEXT NOT NULL,   field_type TEXT, -- text, email, phone, select, checkbox   is_required BOOLEAN DEFAULT FALSE,   options JSONB, -- for select/checkbox options   display_order INT,   created_at TIMESTAMP DEFAULT NOW() );  CREATE TABLE public.form_submissions (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   booking_id UUID REFERENCES bookings(id) ON DELETE CASCADE,   form_id UUID REFERENCES form_templates(id),   customer_id UUID REFERENCES users(id),   submitted_at TIMESTAMP DEFAULT NOW() );  CREATE TABLE public.form_field_values (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   submission_id UUID REFERENCES form_submissions(id) ON DELETE CASCADE,   field_id UUID REFERENCES form_fields(id),   value TEXT );  CREATE TABLE public.favorites (   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),   customer_id UUID REFERENCES users(id) ON DELETE CASCADE,   business_id UUID REFERENCES businesses(id) ON DELETE CASCADE,   created_at TIMESTAMP DEFAULT NOW(),   UNIQUE(customer_id, business_id) );
```

You'll know it's working when...

- All form tables created
- Foreign keys are consistent

Step 22: Create Refunds Table & Seed Data

Refunds track reimbursements. Then seed the database with default categories.

Terminal Command:

```
CREATE TABLE public.refunds (    id UUID PRIMARY KEY DEFAULT
gen_random_uuid(),    payment_id UUID REFERENCES payments(id) NOT NULL,
amount DECIMAL(10, 2) NOT NULL,    reason TEXT,    status TEXT DEFAULT
'pending', -- pending, succeeded, failed    created_at TIMESTAMP DEFAULT
NOW() );  CREATE TABLE public.categories (    id UUID PRIMARY KEY DEFAULT
gen_random_uuid(),    name TEXT UNIQUE NOT NULL,    icon_emoji TEXT,
created_at TIMESTAMP DEFAULT NOW() );
```

You'll know it's working when...

- Refunds and categories tables exist
- Ready for seed data

Step 23: Seed Categories

Insert 40+ service categories so businesses can easily tag their offerings.

Terminal Command:

```
INSERT INTO categories (name, icon_emoji) VALUES ('Hair & Salon', '□'), ('Massage
& Spa', '⚕                                                    '), ('Home Services', '□
('Pet Care', '□'), ('Photography', '□'), ('Beauty & Makeup', '□'), ('Tutoring', '□'),
('Coaching', '⚡                    '), ('Plumbing', '□'), ('Electrical', '□'), ('Cleaning', '□'), ('Childcare', '□
('Gardening', '                    '), ('Dog Walking', '□'), ('Personal Training', '□'), ('Yoga', '□
'□'), ('Pilates', '□'), ('Swimming', '□'), ('Tennis', '□'), ('Music Lessons', '□'), ('Dance Classes',
'                                  '), ('Cooking Classes', '□'), ('Therapy & Counseling', '□'), ('Haircut',
'✂'), ('Beard Trim', '                                    '), ('Waxing', '□'), ('Eyebrows', '□'), ('Pedicure', '□
('Eyelash Extension', '□'), ('Tattoo', '□'), ('Piercing', '□'), ('Dentist', '□'),
('Chiropractor', '□'), ('Acupuncture', '□'), ('Veterinary', '□');
```

You'll know it's working when...

- 40+ categories are inserted
- SELECT COUNT(*) FROM categories returns 40+

Step 24: Set Up Row Level Security (RLS)

RLS ensures users can only access their own data. This is critical for security.

Terminal Command:

```
-- Enable RLS on critical tables: ALTER TABLE public.users ENABLE ROW LEVEL
SECURITY; ALTER TABLE public.user_profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.businesses ENABLE ROW LEVEL SECURITY; ALTER TABLE
public.bookings ENABLE ROW LEVEL SECURITY; ALTER TABLE
public.notifications ENABLE ROW LEVEL SECURITY;  -- Example policy: Users can
only view their own profile CREATE POLICY "Users can view own profile" ON
user_profiles    FOR SELECT USING (user_id = auth.uid());
```

You'll know it's working when...

- RLS is enabled on all critical tables
- No queries fail

Step 25: Create Indices for Performance

Database indices speed up queries on frequently searched columns.

Terminal Command:

```
CREATE INDEX idx_users_email ON public.users(email); CREATE INDEX
idx_user_profiles_user_id ON public.user_profiles(user_id); CREATE INDEX
idx_businesses_owner_id ON public.businesses(owner_id); CREATE INDEX
idx_services_business_id ON public.services(business_id); CREATE INDEX
idx_bookings_customer_id ON public.bookings(customer_id); CREATE INDEX
idx_bookings_business_id ON public.bookings(business_id); CREATE INDEX
idx_bookings_scheduled_start ON public.bookings(scheduled_start); CREATE INDEX
idx_reviews_business_id ON public.reviews(business_id); CREATE INDEX
idx_notifications_user_id ON public.notifications(user_id);
```

You'll know it's working when...

- Indices are created
- Queries on these columns are faster

Step 26: Create Database Functions

Functions automate repetitive tasks like updating booking status history.

Terminal Command:

```
CREATE OR REPLACE FUNCTION public.log_booking_status_change() RETURNS
TRIGGER AS $$ BEGIN   INSERT INTO public.booking_status_history (booking_id,
old_status, new_status, changed_at)   VALUES (NEW.id, OLD.status, NEW.status,
NOW());   RETURN NEW; END; $$ LANGUAGE plpgsql;  CREATE TRIGGER
booking_status_trigger AFTER UPDATE ON public.bookings FOR EACH ROW
WHEN (OLD.status IS DISTINCT FROM NEW.status) EXECUTE FUNCTION
public.log_booking_status_change();
```

You'll know it's working when...

- Function is created
- Booking status changes are logged automatically

Step 27: Create Seed Data Script

A seed script populates test data so you can develop features without manual data entry.

Terminal Command:

```
-- Create seed file: scripts/seed.sql INSERT INTO categories (name, icon_emoji)
VALUES ... -- (from step 23)  INSERT INTO users (email) VALUES
('owner@example.com'), ('customer@example.com');  INSERT INTO businesses
(owner_id, name, city, state) VALUES ((SELECT id FROM users WHERE
email='owner@example.com'),  'Sample Salon', 'San Francisco', 'CA');
```

You'll know it's working when...

- Seed script is created
- Running it inserts test data

Step 28: Validate Database Schema

Verify all tables, foreign keys, and indices are correct before moving to the application layer.

Terminal Command:

```
-- In Supabase SQL Editor, run: SELECT * FROM information_schema.tables WHERE table_schema='public';  -- Count tables: SELECT COUNT(*) FROM information_schema.tables WHERE table_schema='public';
```

You'll know it's working when...

- All ~20 tables are listed
- Foreign key constraints are correct

PART 3: AUTHENTICATION (Steps 29-35)

Set up Supabase Auth so users can sign up, log in, and reset their passwords securely.

Step 29: Configure Supabase Auth Providers

Enable email/password auth and optionally Google/Apple for social login.

Terminal Command:

```
-- In Supabase console: # 1. Go to Authentication > Providers # 2. Enable Email (if not already enabled) # 3. Configure email templates (optional) # 4. For social login: Enable Google (or Apple/GitHub) # 5. Add OAuth credentials from Google Cloud Console
```

You'll know it's working when...

- Email provider is enabled
- Sign-up/login emails are sent

Step 30: Create Auth Context in Next.js

Build an Auth Context that manages user state across your entire Next.js app.

Terminal Command:

```
```

You'll know it's working when...

- AuthProvider wraps _app.tsx
- User state persists across page refreshes

Step 31: Build Sign Up & Login Pages

Create forms for new users to sign up and existing users to log in.

Terminal Command:

```
```

You'll know it's working when...

- Sign up form is displayed
- New users receive confirmation emails

Common Issues:

Email not received:

- Check spam folder or resend verification

Step 32: Implement Forgot Password

Let users reset their password if they forget it.

Terminal Command:

You'll know it's working when...

- Users receive password reset emails
- Reset links work and allow new password entry

Step 33: Create Auth Middleware

Protect pages so only logged-in users can access them.

Terminal Command:

You'll know it's working when...

- Unauthenticated users are redirected to login
- Logged-in users can access /dashboard

Step 34: Build Profile Completion Flow

After sign-up, prompt users to complete their profile (name, phone, etc.).

Terminal Command:

```
-- Check if user has profile: SELECT * FROM user_profiles WHERE user_id = auth.uid();  -- If not, show profile form: -- Collect first_name, last_name, phone, avatar_url  -- Save profile: INSERT INTO user_profiles (user_id, first_name, last_name, phone) VALUES (auth.uid(), 'John', 'Doe', '5551234567');
```

You'll know it's working when...

- Users are prompted for profile info after signup
- Profiles are saved and retrievable

Step 35: Test Auth Flow End-to-End

Manually test the entire auth flow: signup, email verification, login, logout.

Terminal Command:

```
# Test steps: # 1. Sign up with a new email # 2. Check email inbox for verification link # 3. Click verification link # 4. Log in with the same email and password # 5. Profile page should show your data # 6. Test password reset # 7. Log out and verify redirected to login
```

You'll know it's working when...

- Sign up, login, and logout all work
- Email verification is required before login

Common Issues:

Auth state not persisting:

- Check localStorage is enabled in browser

PART 4: BUSINESS DASHBOARD - CORE (Steps 36-50)

Build the web dashboard where business owners manage their profiles, services, availability, and bookings.

Step 36: Set Up Next.js Project with Tailwind & shadcn/ui

Initialize the web app with modern styling and pre-built components.

Terminal Command:

```
# In apps/web folder: npx create-next-app@latest . --typescript --tailwind npm install -D shadcn-ui npx shadcn-ui@latest init
```

You'll know it's working when...

- Next.js dev server runs at http://localhost:3000
- Tailwind classes work
- shadcn/ui components are available

Step 37: Create Dashboard Layout

Build the main layout: sidebar navigation, header, content area.

Terminal Command:

```
```

You'll know it's working when...

- Sidebar and main content render
- Navigation links work

Step 38: Build Business Onboarding Wizard

Step-by-step form for new businesses: info, address, hours, photos.

Terminal Command:

```
-- Step 1: Business Info (name, description) -- Step 2: Address (street, city, state, zip) + Geocoding (latitude/longitude) -- Step 3: Hours (opening/closing times per day) -- Step 4: Photos (upload logo and gallery) -- Step 5: Review & Submit
```

You'll know it's working when...

- Wizard displays step by step
- Data is saved to database after each step

Step 39: Implement Address & Geocoding

Convert addresses to coordinates for location-based search.

Terminal Command:

You'll know it's working when...

- Addresses are geocoded to latitude/longitude
- Coordinates are saved in database

Step 40: Build Profile Editor

Allow owners to edit business information after initial setup.

Terminal Command:

```
-- Page: /dashboard/profile -- Form fields:   - Business name   - Description   - Address   - Phone   - Website   - Logo upload  -- Submit updates to: UPDATE businesses SET name=?, description=?, ... WHERE id=?
```

You'll know it's working when...

- Profile editor loads with current data
- Changes save without errors

Step 41: Create Service Manager (CRUD)

Create, read, update, delete services a business offers.

Terminal Command:

```
-- Page: /dashboard/services -- List all services for this business -- Button to add new service -- Edit/delete buttons on each service  CREATE form inputs: - Service name - Description - Duration (minutes) - Price
```

You'll know it's working when...

- Services list displays all services
- Add/edit/delete all work

Step 42: Build Availability Settings

Owners set their working hours and how many slots per hour.

Terminal Command:

```
-- Page: /dashboard/availability -- Form per staff member -- For each day (Mon-Sun): - Toggle: Working day   - Start time (HH:MM)   - End time (HH:MM)   - Slots per hour (e.g., 2 = 30-min slots)  -- Save to availability_rules table
```

You'll know it's working when...

- Availability form displays and saves
- Slots per hour controls booking granularity

Step 43: Create Calendar View

Show bookings on a calendar so owners can see their schedule at a glance.

Terminal Command:

```
-- Use a library like react-big-calendar or fullcalendar npm install react-big-calendar
-- Query bookings: SELECT * FROM bookings WHERE business_id = ? AND
scheduled_start BETWEEN ? AND ?
```

You'll know it's working when...

- Calendar displays month/week view
- Bookings appear as events on the calendar

Step 44: Build Bookings List & Detail View

Show all bookings and detailed info for each one.

Terminal Command:

```
-- Page: /dashboard/bookings -- List:   - Customer name   - Service   - Scheduled
date/time   - Status (pending, confirmed, completed, cancelled)   - Click to view
detail  -- Detail page:   - All booking info   - Intake form responses (if any)   - Payment
status   - Buttons: Confirm, Complete, Cancel
```

You'll know it's working when...

- Bookings list loads and is sortable
- Detail page shows all data

Step 45: Implement Stripe Connect Onboarding

Connect business bank accounts to Stripe so they get paid.

Terminal Command:

```
```

You'll know it's working when...

- Stripe onboarding link is created
- Business is redirected to Stripe and returns after completion

Step 46: Connect Stripe Account to Backend

Call Stripe API to create account and get link.

Terminal Command:

```
```

You'll know it's working when...

- API endpoint creates Stripe account
- Account ID is saved

Step 47: Build Basic Analytics

Show owner key metrics: total bookings, revenue, ratings.

Terminal Command:

```
-- Page: /dashboard/analytics -- Metrics:   - Total bookings (this month, this year)   -
Total revenue   - Average rating   - Booking trend (line chart)  -- Queries: SELECT
```

```
COUNT(*) FROM bookings WHERE business_id = ? AND scheduled_start > ?
SELECT SUM(total_price) FROM bookings WHERE business_id = ? SELECT
AVG(rating) FROM reviews WHERE business_id = ?
```

You'll know it's working when...

- Analytics page displays key metrics
- Charts render correctly

Step 48: Add Booking Confirmation Logic

When a customer books, the business gets notified and must confirm (or auto-confirm).

Terminal Command:

```
-- Option A: Auto-confirm UPDATE bookings SET status = 'confirmed' WHERE id
= ?;  -- Option B: Manual confirm -- Add button on /dashboard/bookings/[id] -- When
clicked, run UPDATE above -- Send confirmation email to customer
```

You'll know it's working when...

- Bookings can be confirmed
- Customers are notified of confirmation

Step 49: Implement Email Notifications via Resend

Send emails to customers when bookings are confirmed, upcoming, etc.

Terminal Command:

```

```

You'll know it's working when...

- Confirmation emails are sent
- Emails arrive in customer inbox

Step 50: Test Full Booking Workflow

End-to-end test: business setup, service creation, and mock booking.

Terminal Command:

```
# Manual test: # 1. Sign up as business owner # 2. Complete onboarding (name,
address, hours) # 3. Create a service # 4. Set availability # 5. Manually insert a
booking via SQL (or wait for mobile app) # 6. View booking on dashboard # 7. Click
Confirm # 8. Check for confirmation email
```

You'll know it's working when...

- All steps work without errors
- Booking appears on calendar

PART 5: BUSINESS DASHBOARD - ADVANCED (Steps 51-58)

Build advanced features: intake forms, staff management, reviews, and settings.

Step 51: Build Intake Form Builder

Allow businesses to create custom questionnaires for customers (e.g., massage: health history, injury status).

Terminal Command:

```
-- Page: /dashboard/forms -- List existing forms -- Button: Create New Form -- Form name input -- Drag-and-drop or list to add fields
```

You'll know it's working when...

- Form builder interface is intuitive
- Forms save with fields

Step 52: Create Form Field Configuration UI

Interface to add fields (text, email, phone, select, checkbox) with validation.

Terminal Command:

```
-- For each field:   - Field name   - Field type (text, email, phone, select, checkbox)   - Required toggle   - Options (for select/checkbox)   - Display order
```

You'll know it's working when...

- Fields can be added/edited/deleted
- Field types are validated

Step 53: Build Form Preview & Testing

Let business owners see how forms look to customers before going live.

Terminal Command:

```
-- Page: /dashboard/forms/[id]/preview -- Render form as customer would see it -- Fill it out with dummy data -- Submit button (test-only)
```

You'll know it's working when...

- Form preview renders all fields
- Submission works (no actual save)

Step 54: Create Form Submissions Viewer

View all submitted form responses for a given form or booking.

Terminal Command:

```
-- Page: /dashboard/forms/[id]/submissions -- Table of submissions:   - Customer name   - Submission date   - View responses -- Click to see all field values
```

You'll know it's working when...

- Submissions list displays
- Detail view shows all responses

Step 55: Build Staff Manager

Create, edit, list team members and assign them to services.

Terminal Command:

```
-- Page: /dashboard/staff -- Add staff:  - Name  - Email  - Phone  - Active toggle -- Edit/delete buttons
```

You'll know it's working when...

- Staff list displays
- Add/edit/delete work

Step 56: Implement Staff-Service Assignments

Link staff to services they can provide.

Terminal Command:

```
-- Page: /dashboard/staff/[id]/services -- Show all services -- Checkboxes to assign/remove services -- Save to staff_services table
```

You'll know it's working when...

- Staff can be assigned to multiple services
- Services can have multiple staff

Step 57: Create Reviews Management Page

View customer reviews and respond to them.

Terminal Command:

```
-- Page: /dashboard/reviews -- List reviews:  - Customer name  - Rating (stars)  - Comment  - Date  - Business response (if any) -- Button: Write Response -- Response form
```

You'll know it's working when...

- Reviews display
- Business can write responses

Step 58: Build Settings Page

Business hours, notification preferences, payment settings, etc.

Terminal Command:

```
-- Page: /dashboard/settings -- Sections:  - Business hours  - Notification preferences (email, SMS)  - Payment settings (Stripe)  - Cancel/delete business
```

You'll know it's working when...

- Settings form loads
- Changes save correctly

PART 6: CONSUMER MOBILE APP - SETUP (Steps 59-63)

Initialize the React Native mobile app using Expo for iOS and Android.

Step 59: Create Expo Project

Set up the foundation for the React Native mobile app.

Terminal Command:

```
cd apps/mobile npx create-expo-app serva cd serva npm install
```

You'll know it's working when...

- Project folder is created
- npm install completes

Step 60: Install Core Mobile Dependencies

Add libraries for navigation, state management, and networking.

Terminal Command:

```
npm install @react-navigation/native @react-navigation/bottom-tabs @react-navigation/stack npm install expo-location expo-image-picker @react-native-async-storage/async-storage npm install zustand axios
```

You'll know it's working when...

- All packages install without errors

Step 61: Set Up React Navigation (Tabs + Stacks)

Create tab-based navigation (Home, Search, Bookings, Profile).

Terminal Command:

```
```

You'll know it's working when...

- Navigation structure is in place
- Tabs display and switch

Step 62: Configure Supabase Client for React Native

Initialize Supabase with proper AsyncStorage for session persistence.

Terminal Command:

```
```

You'll know it's working when...

- Supabase client initializes
- Auth state persists after app restart

Step 63: Build Mobile Auth Flow & Theme

Create login/signup screens and establish a consistent design theme.

Terminal Command:

```
-- Pages:   - /app/auth/login.tsx   - /app/auth/signup.tsx   - /app/auth/forgot-
password.tsx  -- Theme:   - Navy primary color (#1A1F36)   - Blue accent (#4A6CF7)
- Use consistent fonts and spacing
```

You'll know it's working when...

- Auth screens render
- Theme is consistent across app

PART 7: CONSUMER MOBILE APP - FEATURES (Steps 64-82)

Build customer-facing features: search, business profiles, booking, payments, reviews.

Step 64: Create Home Screen

Display featured businesses, categories, and quick access to search.

Terminal Command:

```
-- Components:   - Search bar (tap to go to Search screen)   - Category carousel (horizontal scroll)   - Featured businesses (grid or list)   - Bottom tabs
```

You'll know it's working when...

- Home screen displays
- Categories and businesses load

Step 65: Build Search Screen & Full-Text Search

Let customers find businesses by name, service, or city.

Terminal Command:

```
```

You'll know it's working when...

- Search results load
- Filtering by name/city works

Step 66: Implement Geo-Location Search

Search by radius: find nearby businesses based on user location.

Terminal Command:

```
```

You'll know it's working when...

- User location is requested
- Nearby businesses are returned in order of distance

Step 67: Create Business Profile Screen

Display all info about a business: photos, services, hours, reviews.

Terminal Command:

```
-- Sections:   1. Header (logo, name, rating)   2. Photos (carousel)   3. About section (description, hours, address)   4. Services (list with prices/durations)   5. Reviews (scrollable list)   6. Contact (phone, website)
```

You'll know it's working when...

- Profile loads with all data
- Sections scroll smoothly

Step 68: Build Services/Reviews/Photos/Info Tabs

Organize business info into tab-based view.

Terminal Command:

```
-- Tabs:   - Services: List of services with prices   - Reviews: Customer reviews with ratings   - Photos: Gallery of business photos   - Info: Hours, address, contact info
```

You'll know it's working when...

- Tabs switch without errors
- Data loads in each tab

Step 69: Create Calendar & Date/Time Picker

Let customers choose a date and available time slot for booking.

Terminal Command:

```
-- Components:   - Calendar (month view, tap to select date)   - Time slots (for selected date, disabled if fully booked)   - Display staff member (if applicable)  -- Query available slots: SELECT * FROM availability_rules WHERE staff_id = ?  FILTER out booked slots
```

You'll know it's working when...

- Calendar displays and allows date selection
- Time slots update when date changes

Step 70: Implement Real-Time Availability & Slot Locking

Prevent double-bookings by temporarily locking slots.

Terminal Command:

```
-- When customer selects a slot:   1. Lock it for 10 minutes (optimistic lock)   2. If they don't book, release after timeout   3. Prevent other users from booking same slot
```

You'll know it's working when...

- Slots can be booked without conflicts
- Locked slots show as unavailable to others

Step 71: Build Dynamic Intake Form Renderer

Display business's custom intake form to customer during booking.

Terminal Command:

```

```

You'll know it's working when...

- Intake form displays with correct field types
- Form submission saves responses

Step 72: Implement Stripe Payment Sheet

Integrate Stripe's mobile payment UI for seamless checkout.

Terminal Command:

```
npm install @stripe/stripe-react-native  -- In booking screen: -- After form submission, initialize Stripe payment sheet -- Let customer enter card details -- Confirm payment via Stripe
```

You'll know it's working when...

- Payment sheet opens
- Payment is processed

Step 73: Build Booking Confirmation Screen

Show confirmation after successful payment with booking details.

Terminal Command:

```
-- Display:   - Booking confirmation number   - Service name, date, time   - Total cost   - Business contact info   - Add to calendar button
```

You'll know it's working when...

- Confirmation shows all details
- Add to calendar option works

Step 74: Create My Bookings Screen

Show all bookings (upcoming, past, cancelled) for the logged-in customer.

Terminal Command:

```
-- List bookings:   - Upcoming (sorting by date ascending)   - Past (completed bookings)   - Cancelled  -- For each: business name, service, date/time, status
```

You'll know it's working when...

- Bookings list loads
- Filtering by status works

Step 75: Build Booking Detail Screen

Full details of a booking with options to cancel or reschedule.

Terminal Command:

```
-- Show:   - All booking details   - Business info   - Directions button (Google Maps)   - Buttons: Cancel, Reschedule, Directions  -- Cancel flow: select reason, confirm -- Reschedule: go back to calendar picker
```

You'll know it's working when...

- Booking detail displays all info

- Cancel/reschedule buttons work

Step 76: Implement Cancellation Logic

Allow customers to cancel bookings (with business rules on deadlines).

Terminal Command:

```
-- Check: Can this booking be cancelled?  - Business policy: e.g., must cancel 24 hours before  - If allowed: DELETE or UPDATE status to cancelled  - Refund payment if applicable  - Send cancellation email
```

You'll know it's working when...

- Customers can cancel bookings
- Refunds are processed

Step 77: Build Rescheduling Feature

Allow customers to change booking date/time.

Terminal Command:

```
-- Process:  1. Show calendar with available slots  2. Select new date/time  3. Update booking  4. Send email to business and customer
```

You'll know it's working when...

- Bookings can be rescheduled
- Notifications are sent

Step 78: Create Write Review Screen

After booking completion, prompt customer to leave a review.

Terminal Command:

```
-- Show:  - Business name & photo  - Star rating (1-5)  - Text comment field (optional)  - Submit button  -- Trigger: 1 day after booking completion
```

You'll know it's working when...

- Review form displays
- Reviews are saved

Step 79: Build User Profile & Settings Screen

Let customers manage their profile and app preferences.

Terminal Command:

```
-- Profile section:  - Name, email, phone  - Avatar upload  - Edit button  -- Settings:  - Notification preferences (email, push, SMS)  - Language/timezone  - Delete account
```

You'll know it's working when...

- Profile loads with user data
- Changes save

Step 80: Implement Favorites / Saved Businesses

Let customers bookmark businesses for quick access.

Terminal Command:

```
-- Icon: heart on business profile -- When tapped: INSERT into favorites table -- Show favorites on home screen or dedicated tab
```

You'll know it's working when...

- Favorites can be added/removed
- Favorites list displays

Step 81: Add Push Notifications Setup

Prepare infrastructure for sending push notifications (implemented in Part 8).

Terminal Command:

```
-- Install library: npm install expo-notifications  -- Store push token in database: const token = await Notification.getPermissionsAsync();
```

You'll know it's working when...

- Push notification library is installed

Step 82: Test Full Booking Flow on Device

End-to-end test on your phone with Expo Go.

Terminal Command:

```
# Steps: # 1. expo start # 2. Scan QR code with phone camera # 3. App opens in Expo Go # 4. Log in as customer # 5. Search for a business # 6. View profile # 7. Book a service # 8. Make payment # 9. See confirmation # 10. View in My Bookings
```

You'll know it's working when...

- App loads on phone
- All booking steps work smoothly

Common Issues:

Network error:

- Ensure phone is on same WiFi as dev computer

PART 8: NOTIFICATIONS (Steps 83-88)

Set up email, SMS, and push notifications to keep users informed.

Step 83: Configure Expo Push Notifications

Enable push notifications through Expo's servers.

Terminal Command:

```
# Request permission const { status } = await Notification.requestPermissionsAsync();  # Register for push notifications const token = (await Notification.getDevicePushTokenAsync()).data;  # Save to database await supabase.from('users').update({ push_token: token }).eq('id', user.id);
```

You'll know it's working when...

- Push token is generated and saved
- Permission dialog appears on first launch

Step 84: Set Up Resend Email Templates

Create reusable email templates for confirmations, reminders, etc.

Terminal Command:

```
-- Email templates:   1. Booking confirmation   2. Booking reminder (24h before)   3. Cancellation confirmation   4. Review request   5. Review response  -- Use Resend dashboard or API to create
```

You'll know it's working when...

- Email templates are created in Resend
- Emails are sent successfully

Step 85: Configure Twilio SMS

Send SMS reminders and updates to users.

Terminal Command:

```
```

You'll know it's working when...

- SMS is sent successfully
- Messages arrive on phone

Step 86: Implement Reminder Scheduling

Automatically send reminders 24 hours before bookings.

Terminal Command:

```
-- Cron job (e.g., using node-cron or AWS Lambda): -- Every hour, find bookings scheduled for tomorrow -- Send reminder email/SMS  SELECT * FROM bookings WHERE scheduled_start BETWEEN NOW() + '23 hours' AND NOW() + '24 hours' AND reminder_sent = FALSE;
```

You'll know it's working when...

- Scheduled reminders are sent
- Reminders don't duplicate

Step 87: Build In-App Notification Center

Display notifications within the app (bookings, reviews, updates).

Terminal Command:

```
-- Mobile screen: /app/notifications -- Query: SELECT * FROM notifications WHERE user_id = ? ORDER BY created_at DESC -- Mark as read when tapped -- Clear old notifications (7+ days)
```

You'll know it's working when...

- Notifications display in app
- Tapping marks as read

Step 88: Test Notification Workflows

End-to-end test: trigger booking, receive email, SMS, and push notification.

Terminal Command:

```
# Test: # 1. Create a booking via API # 2. Check email inbox for confirmation # 3. Check SMS (if number provided) # 4. Check push notification on phone # 5. Verify in-app notification center
```

You'll know it's working when...

- All notifications are received
- Content is correct

PART 9: ADMIN PANEL (Steps 89-95)

Build administrative tools for managing businesses, users, and content moderation.

Step 89: Create Admin Layout & Auth Guard

Restrict admin pages to authorized admins only.

Terminal Command:

```
```

You'll know it's working when...

- Admin pages require role check
- Non-admins are redirected

Step 90: Build Admin Dashboard

Overview of key metrics: total users, businesses, revenue, bookings.

Terminal Command:

```
-- Page: /admin -- Metrics:  - Total users (all time, this month)  - Total businesses (pending approval, approved)  - Total revenue  - Total bookings (this month)  -- Charts:  - Bookings trend (line chart)  - Revenue trend
```

You'll know it's working when...

- Dashboard displays all metrics
- Charts render

Step 91: Create Business Approval Queue

View pending businesses and approve or reject them.

Terminal Command:

```
-- Page: /admin/businesses -- List: businesses WHERE is_approved = FALSE -- For each:  - Business name, owner name  - Created date  - Quick view (photos, info)  - Buttons: Approve, Reject
```

You'll know it's working when...

- Pending businesses list loads
- Approval/rejection buttons work

Step 92: Implement Business Approve/Reject Logic

Approve or reject businesses with optional feedback.

Terminal Command:

```
-- Approve: UPDATE businesses SET is_approved = TRUE WHERE id = ?; SEND email to owner: 'Your business is approved!'  -- Reject: UPDATE businesses SET
```

```
is_approved = FALSE WHERE id = ?; SEND email to owner: 'Your business was rejected. Reason: ...'
```

You'll know it's working when...

- Businesses can be approved/rejected
- Owners receive emails

Step 93: Build User Management Page

Search, view, and manage user accounts.

Terminal Command:

```
-- Page: /admin/users -- Search by email -- List: email, name, role, created date, status -- Actions: View profile, reset password, deactivate, delete
```

You'll know it's working when...

- User list loads
- Search works

Step 94: Create Review Moderation Page

Flag and remove inappropriate reviews.

Terminal Command:

```
-- Page: /admin/reviews -- List: all reviews with ratings, comments -- Flag suspicious reviews -- Remove if violated policy
```

You'll know it's working when...

- Reviews list displays
- Moderation actions work

Step 95: Build Category Management

Add, edit, delete service categories.

Terminal Command:

```
-- Page: /admin/categories -- List all categories -- Add new: name, emoji -- Edit, delete buttons
```

You'll know it's working when...

- Categories list loads
- Add/edit/delete work

PART 10: TESTING & LAUNCH (Steps 96-108)

Test the complete system, deploy to production, and submit to app stores.

Step 96: Write Integration Tests for Booking Flow

Automated tests to verify the entire booking process.

Terminal Command:

```
```

You'll know it's working when...

- Tests run and pass
- Booking flow is verified

Step 97: Write Integration Tests for Payment Flow

Test payment creation and confirmation.

Terminal Command:

```
-- Test:  1. Create booking  2. Create Stripe payment intent  3. Confirm payment
4. Verify booking status changes to 'confirmed'  5. Verify payment status is
'succeeded'
```

You'll know it's working when...

- Payment tests pass
- Stripe integration is verified

Step 98: Implement End-to-End Testing with Playwright

Automated browser testing to simulate real user interactions.

Terminal Command:

```
npm install -D @playwright/test  -- Write tests for:  - Sign up flow  - Business
dashboard  - Customer search and booking  - Admin panel
```

You'll know it's working when...

- Playwright tests run
- All user flows are verified

Step 99: Set Up Error Tracking with Sentry

Catch errors in production and get alerts.

Terminal Command:

```
npm install @sentry/nextjs @sentry/react-native  -- Initialize in pages/_app.tsx and
app.json -- Errors are automatically reported to Sentry dashboard
```

You'll know it's working when...

- Errors appear in Sentry
- Alerts are configured

Step 100: Deploy Web App to Vercel

Push Next.js app to production with continuous deployment.

Terminal Command:

```
# 1. Push code to GitHub (main branch) # 2. Go to vercel.com and import GitHub repo # 3. Vercel auto-builds and deploys # 4. Set environment variables (Supabase keys, Stripe, etc.) # 5. App is live at your-domain.vercel.app
```

You'll know it's working when...

- App is deployed
- Live URL is accessible

Step 101: Connect Custom Domain to Vercel

Point your domain to the Vercel deployment.

Terminal Command:

```
# In Vercel dashboard: # 1. Settings > Domains # 2. Add domain # 3. Update DNS records (CNAME) at your domain registrar # 4. Verify DNS propagation (can take 24h) # 5. Domain is live
```

You'll know it's working when...

- Custom domain resolves to Vercel app

Step 102: Build Mobile App with EAS Build

Use Expo Application Services to build iOS and Android binaries.

Terminal Command:

```
# Install EAS CLI: npm install -g eas-cli  # Login: eas login  # Build: eas build --platform ios eas build --platform android
```

You'll know it's working when...

- EAS build completes
- IPA and APK files are generated

Step 103: Test iOS Build with TestFlight

Distribute iOS build to testers via Apple TestFlight.

Terminal Command:

```
# After EAS build completes: # 1. App appears in App Store Connect # 2. Add TestFlight testers (email addresses) # 3. Testers receive link to download app # 4. Test on their iPhones
```

You'll know it's working when...

- App appears in TestFlight

- Testers can download and test

Step 104: Test Android Build with Google Play Console

Distribute Android build to testers via Play Store internal testing.

Terminal Command:

```
# After EAS build completes: # 1. App appears in Google Play Console # 2. Add testers to internal testing group # 3. Testers get Play Store link # 4. Test on their Android devices
```

You'll know it's working when...

- App appears in Play Store internal testing
- Testers can download

Step 105: Submit App to App Store

Go through App Store submission process (review takes 24-48 hours).

Terminal Command:

```
# In App Store Connect: # 1. Complete app info (screenshots, description, keywords) # 2. Set age rating and content # 3. Configure pricing # 4. Submit for review # 5. Wait for Apple approval # 6. App goes live when approved
```

You'll know it's working when...

- App is submitted to App Store
- Approval is received

Step 106: Submit App to Google Play

Launch Android app on Google Play Store.

Terminal Command:

```
# In Google Play Console: # 1. Complete store listing (screenshots, description) # 2. Set content rating # 3. Configure pricing # 4. Submit for review # 5. Approval usually takes 2-4 hours # 6. App goes live when approved
```

You'll know it's working when...

- App is submitted to Google Play
- Approval is received

Step 107: Set Up Analytics

Track user behavior and app usage with Google Analytics or Mixpanel.

Terminal Command:

```
npm install analytics @react-native-google-analytics-bridge  -- Track events:   - User sign up   - Booking created   - Payment succeeded   - Review submitted
```

You'll know it's working when...

- Analytics dashboard shows data

- Key events are tracked

Step 108: Create & Execute Launch Checklist

Final verification before public launch.

Terminal Command:

```
Checklist: □ All tests pass □ No critical errors in Sentry □ Web app is live at custom domain □ iOS app is on App Store □ Android app is on Google Play □ Email/SMS sending works □ Stripe is connected and processing payments □ Database backups are configured □ Analytics are tracking □ Support email is monitored □ Documentation is complete □ Marketing materials are ready □ Launch date is set
Actions: 1. Announce launch on social media 2. Email early users 3. Monitor error tracking 4. Respond to user feedback
```

You'll know it's working when...

- All checklist items are completed
- Launch is successful